A D PATEL INSTITUTE OF TECHNOLOGY
Department of Information Technology

A MINI PROJECT REPORT ON

TYPING SPEED TEST APPLICATION

(Measuring Typing Speed in WPM, Accuracy and Time Taken Using Python)

Subject: Python Programming

Submitted by
Meet Shukla
Enrollment No.: 12402080601029
Department: IT-A

Under the guidance of
Nayan Mali

Information Technology
A D Patel Institute of Technology

Academic Year 2026 – 27

Certificate

This is to certify that the mini project report entitled **"Typing Speed Test Application"** is a bonafide record of the work carried out by **Meet Shukla** (Enrollment No. 12402080601029), a student of the Department of Information Technology (IT-A), A D Patel Institute of Technology, CVM University, Vallabh Vidyanagar, for the subject **Python Programming** during the academic year 2026 – 27.

The work has been completed under my guidance and supervision. To the best of my knowledge, the report has not been submitted to any other institute or university for the award of any degree, diploma or certificate.

Nayan Mali	**Head of Department**
Project Guide	Department of Information Technology
Department of Information Technology	A D Patel Institute of Technology

Date: _____

Place: Vallabh Vidyanagar

Acknowledgement

I would like to express my sincere gratitude to everyone who helped me in completing this mini project on the **Typing Speed Test Application**.

First of all, I thank my guide, **Nayan Mali**, for his valuable guidance, constant encouragement and useful suggestions during every stage of the project. I am also thankful to the Head of the Department of Information Technology and to all the faculty members of A D Patel Institute of Technology for providing the laboratory facilities and the learning environment in which this project could be carried out.

I thank the official Python documentation, which I used as a reference. Finally, I thank my classmates, friends and family for their support and motivation.

Meet Shukla
Enrollment No.: 12402080601029

Abstract

Typing is a basic skill for every student and professional, and it is improved only through regular practice and feedback. A typing speed test gives that feedback by showing how fast and how accurately a person types a given text.

This project presents a Typing Speed Test Application developed in Python. The program shows a sample passage, records the time when the user starts typing, captures the typed text and then calculates three results: the typing speed in Words Per Minute (WPM), the accuracy as a percentage, and the time taken. It uses only the built-in `time` module, string processing and basic arithmetic, and it handles empty input and zero elapsed time safely to avoid division by zero. A retry option lets the user practise again.

Keywords: typing speed test, Python, WPM, accuracy, time module, string processing.

1. Introduction

1.1 Overview

A typing speed test application measures how fast and how accurately a user types a given text. Speed alone is not enough, because a fast typist who makes many mistakes wastes time on corrections; therefore the application reports both speed and accuracy, together with the time taken for the test.

1.2 Need for the Project

Many people want to improve their typing but do not have a simple tool that gives instant and clear numbers. A small console program is enough for this purpose. It also gives a student a practical exercise in Python: reading input, measuring time, comparing strings, doing calculations and printing formatted results.

2. Problem Statement and Objectives

2.1 Problem Statement

Users need a simple way to measure and improve their typing speed and accuracy.

2.2 Objectives

- To calculate typing speed in Words Per Minute (WPM).
- To measure typing accuracy as a percentage.
- To display the time taken to type the given text.
- To provide a simple and user-friendly interface.
- To encourage regular typing practice through a retry option.

3. Technologies Used and Requirements

3.1 Technologies Used

Table 1: Role of each technology

Technology	Role in the project
Python	Main programming language of the application
time module	Tracks the test duration (start time and end time)
String processing	Compares the typed text with the given passage character by character
Basic calculations	Computes WPM, accuracy and elapsed time
Tkinter (optional)	Graphical user interface, only if a GUI version is built; this report uses the console version

3.2 Features of the Application

- Displays a sample passage for typing.
- Starts and tracks the typing timer.
- Calculates typing speed in WPM and accuracy as a percentage.
- Displays the final results after typing.
- Provides a restart (retry) option.

3.3 System Requirements

Table 2: Hardware and software requirements

Component	Specification
Processor / RAM	Any modern processor; 4 GB RAM or more
Operating system	Windows, Linux or macOS
Software	Python 3.8 or newer (no extra library needed for the console version)
Code editor	Visual Studio Code, IDLE or any text editor

4. Working Methodology and Algorithm

4.1 Algorithm

Step 1. Launch the application.

Step 2. Display the sample passage to the user.

Step 3. Start the timer when typing begins.

Step 4. Capture the typed text of the user.

Step 5. Stop the timer and calculate the elapsed time, WPM and accuracy.

Step 6. Display the final results and ask whether the user wants to try again.

4.2 Flowchart

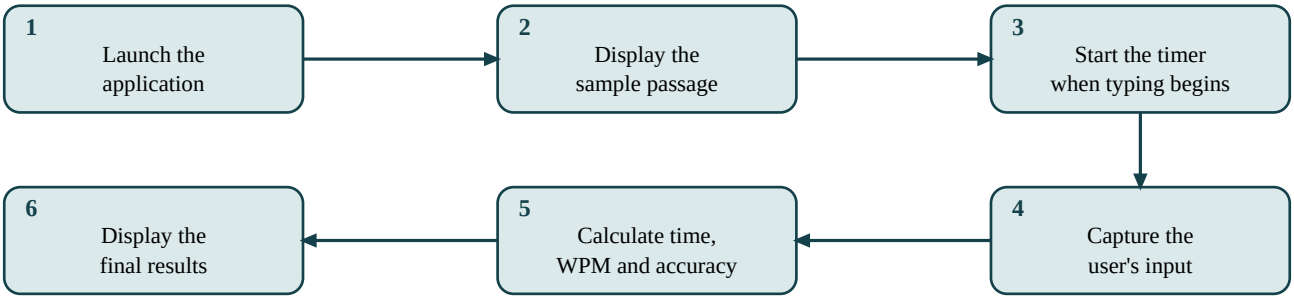


Figure 1: Workflow of the Typing Speed Test Application

4.3 Formulas Used

Table 3: Formulas for the results

Result	Formula	Explanation
Typing speed	WPM = (Number of characters typed ÷ 5) ÷ Time in minutes	One word is taken as 5 characters, the usual standard for typing tests
Accuracy	Accuracy (%) = (Correctly typed characters ÷ Total characters typed) × 100	A typed character is correct if it equals the character at the same position in the passage
Time taken	Elapsed time = End time – Start time	Measured in seconds with time.time(); minutes = seconds ÷ 60

Worked example. If the user types the 23-character passage “Practice makes perfect.” correctly in 12 seconds, then minutes = 12 ÷ 60 = 0.2, words = 23 ÷ 5 = 4.6, so WPM = 4.6 ÷ 0.2 = 23. All 23 characters match, so accuracy = 23 ÷ 23 × 100 = 100%.

4.4 Safe Handling of Special Cases

- **Empty input:** if nothing is typed, WPM and accuracy are both set to 0, so there is no division by zero in the accuracy formula.
- **Zero elapsed time:** if the elapsed time is not greater than zero, WPM is set to 0 instead of dividing by zero.
- **Extra characters:** characters typed beyond the length of the passage have nothing to match, so they count as incorrect and lower the accuracy.
- **Reusable functions:** the calculations are written as two small functions, `calculate_wpm()` and `calculate_accuracy()`, so that they can be tested separately from the keyboard input.

5. Implementation

The application is a single Python file, `typing_test.py`. Blank lines are omitted in the listing to save space. Run it with `python typing_test.py`.

5.1 Source Code (typing_test.py)

```
import random
import time
PASSAGES = [
    "Practice makes perfect.",
    "The quick brown fox jumps over the lazy dog.",
    "Python is simple and powerful.",
]
def calculate_wpm(typed, seconds):
    minutes = seconds / 60
    if not typed or minutes <= 0:
        return 0.0
    return (len(typed) / 5) / minutes
def calculate_accuracy(typed, passage):
    if not typed:
        return 0.0
    correct = sum(a == b for a, b in zip(typed, passage))
    return correct / len(typed) * 100
def run_test():
    passage = random.choice(PASSAGES)
    print("\nType the following text:\n")
    print(passage)
    input("\nPress Enter to start...")
    start = time.time()
    typed = input("> ")
    elapsed = time.time() - start
    wpm = calculate_wpm(typed, elapsed)
    accuracy = calculate_accuracy(typed, passage)
    print(f"\nTyping Speed: {wpm:.0f} WPM")
    print(f"Accuracy: {accuracy:.0f}%")
    print(f"Time Taken: {elapsed:.0f} seconds")
def main():
    print("=== Typing Speed Test ===")
    while True:
        run_test()
        if input("\nTry again? (y/n): ").strip().lower() != "y":
            print("Thank you for practising!")
            break
if __name__ == "__main__":
    main()
```

5.2 Explanation of the Code

- **PASSAGES:** a list of sample texts; `random.choice()` picks one for each test.
- **calculate_wpm():** returns 0 for empty input or non-positive time; otherwise applies (characters ÷ 5) ÷ minutes.
- **calculate_accuracy():** `zip()` pairs the typed and the original characters, `sum()` counts the matches, and the count is divided by the number of typed characters.
- **run_test():** shows the passage, records `time.time()` before and after `input()`, and prints the three results using f-strings.
- **main():** repeats the test until the user answers anything other than “y”.

5.3 Sample Output

Output of a run with the passage typed correctly and the clock fixed at 12 seconds, matching the worked example in Section 4.3.

```
=== Typing Speed Test ===
Type the following text:
Practice makes perfect.
Press Enter to start...
> Practice makes perfect.
Typing Speed: 23 WPM
Accuracy: 100%
Time Taken: 12 seconds
Try again? (y/n): n
Thank you for practising!
```

6. Testing and Results

The two calculation functions were run on eight test cases using the passage “Practice makes perfect.” (23 characters), and the printed values were compared with hand calculations. Table 4 shows the results.

Table 4: Test cases and results

No.	Test	Typed text / time	WPM	Accuracy	Result
1	Exact text	Full passage, 12 s	23.0	100.0%	Pass
2	One wrong character	“Practise…”, 12 s	23.0	95.7%	Pass
3	Empty input	(nothing), 10 s	0.0	0.0%	Pass
4	Zero elapsed time	Full passage, 0 s	0.0	100.0%	Pass
5	Completely wrong text	23 wrong characters, 12 s	23.0	13.0%	Pass
6	Shorter input	“Practice” (8 chars), 6 s	16.0	100.0%	Pass
7	Longer than passage	Passage + “ Extra”, 15 s	23.2	79.3%	Pass
8	Known benchmark	175 correct characters, 60 s	35.0	100.0%	Pass

Observations. All cases matched the expected values. In test 4 the speed is safely 0 instead of an error. In test 5 only the two spaces and the full stop happen to match, giving 13%. Test 6 shows a limitation: accuracy is measured only on the characters typed, so a short but correct input scores 100%. Test 8 reproduces the 35 WPM figure used as the example in the project presentation.

7. Advantages and Limitations

Advantages	Limitations
Simple, readable code using only built-in Python.	Console version only; no graphical interface yet.
Gives speed, accuracy and time in one run.	Accuracy compares characters by position, so one missing letter shifts and penalises all later characters.
Safe against empty input and zero time.	Timing includes the time taken by the Enter key; there is no live countdown timer.
Calculations are separate functions and easy to test.	No record of past scores.

8. Future Scope

- Multiple difficulty levels and longer passages.
- Personal best scores and progress history.
- User profiles and leaderboards.
- Multilingual typing tests and an improved graphical interface (Tkinter).

9. Conclusion

The Typing Speed Test Application provides a simple method to evaluate typing speed and accuracy. It demonstrates Python programming, time measurement, string processing and basic calculations, and the tests show correct results for normal, boundary and empty inputs. With the improvements listed above, it can grow into a fuller typing practice tool.

References

[1] Python Software Foundation, “time – Time access and conversions”, Python documentation. https://docs.python.org/3/library/time.html
[2] Python Software Foundation, “Built-in Functions: zip(), sum()”. https://docs.python.org/3/library/functions.html
[3] Python Software Foundation, “tkinter – Python interface to Tcl/Tk”. https://docs.python.org/3/library/tkinter.html